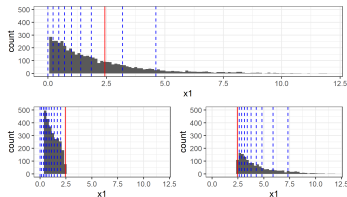
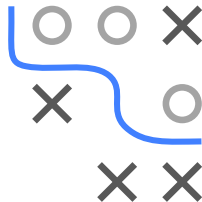


Introduction to Machine Learning

Boosting

Gradient Boosting: XGBoost

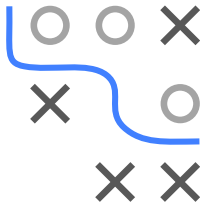


Learning goals

- Overview over XGB
- Regularization in XGB
- Approximate split finding

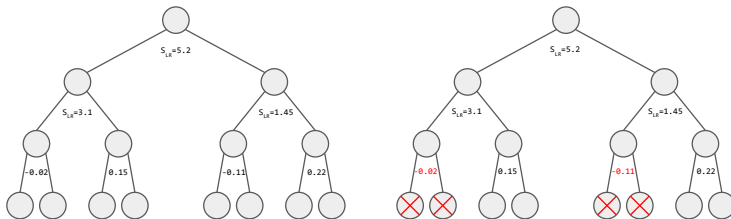
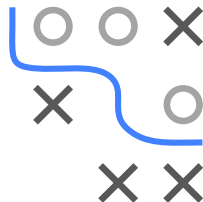
XBG - EXTREME GRADIENT BOOSTING I

- Open-source and scalable tree boosting system
- Efficient implementation in *C++* with interfaces to many other programming languages
- Parallel approximate split finding
- Additional regularization techniques
- Feature and data subsampling
- Cluster and GPU support
- Highly optimized and often achieves top performance in benchmarks – if properly tuned



TREE GROWING I

- Grown to max depth
- Fully expanded and leaves split even if no improvement
- At the end, each split that did not improve risk is pruned



SUBSAMPLING I

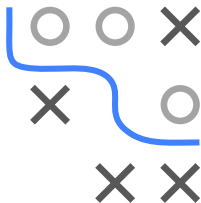
Data Subsampling: XGB uses stochastic GB.

Feature Subsampling: Similar to `mtry` in a random forest only a random subset of features is used for split finding.

The fraction of features for a split can be randomly sampled for each

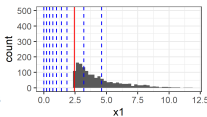
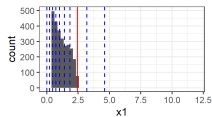
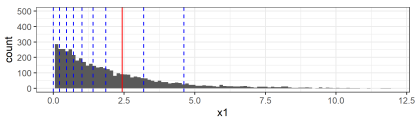
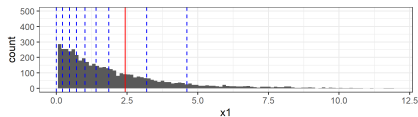
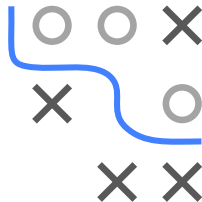
- 1 tree
- 2 level of a tree
- 3 split

Feature subsampling speeds up training even further and can create a more diverse ensemble that often performs better.

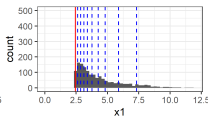
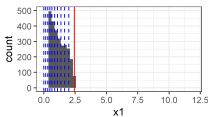


APPROXIMATE SPLIT-FINDING ALGORITHMS I

- Speeds up tree building for large data
- Considers not all, but only I splits per feature
- Usually percentiles of the empirical distribution of each feature
- Computed once (global) or recomputed after each split (local)
- Called **Histogram-based Gradient Boosting**



Global

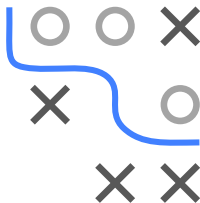


Local

Blue lines are percentiles and red = selected split

PARALLELISM AND GPU COMPUTATION I

- GB is inherently sequential, not easy to parallelize
- **But:** Building of BLs can be parallelized
- Data sort and split eval in different branches of tree BLs can be computed in parallel by using efficient block data structures
- Can also gain huge speed-up by moving from CPU to GPU



OVERVIEW OF IMPORTANT HYPERPARAMETERS

I

HP (as named in software)	Type	Typical Range	Trafo	Default	Description
<code>eta</code>	R	$[-4, 0]$	10^x	0.3	learning rate (also called ν) shrinks contribution of each boosting update
<code>nrounds</code>	I	$\{1, \dots, 5000\}$	–	–	number of boosting iterations. Can also be optimized with early stopping.
<code>gamma</code>	R	$[-7, 6]$	2^x	0	minimum loss reduction required to make a further partition on a leaf node of the tree
<code>max_depth</code>	I	$\{1, \dots, 20\}$	–	6	maximum depth of a tree
<code>colsample_bytree</code>	R	$[0.1, 1]$	–	1	subsample ratio of columns for each tree
<code>colsample_bylevel</code>	R	$[0.1, 1]$	–	1	subsample ratio of columns for each depth level
<code>lambda</code>	R	$[-10, 10]$	2^x	1	$L2$ regularization term on weights
<code>alpha</code>	R	$[-10, 10]$	2^x	0	$L1$ regularization term on weights
<code>subsample</code>	R	$[0.1, 1]$	–	1	subsample ratio of the training instances

